tt_um_santhosh_rvsoc: User and Application Guide

A RISC-V microcontroller on Tiny Tapeout SKY 26d: how it behaves, how to run it in simulation, on the PYNQ-Z2 FPGA and on the real chip, how to program it, and what to build with it.

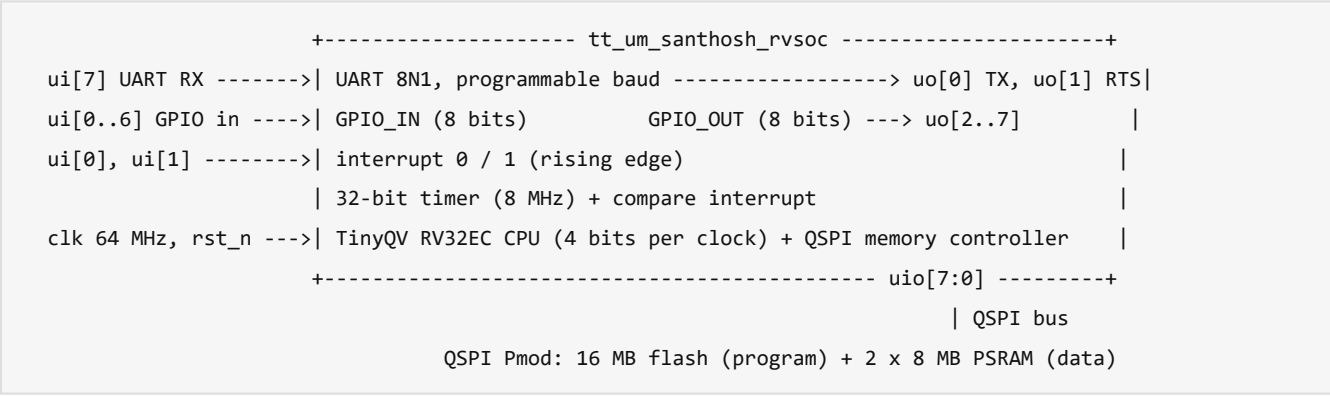
Item	Details
Design	`tt_um_santhosh_rvsoc`, 2x2 Tiny Tapeout tiles, SkyWater 130 nm (sky130A)
CPU	TinyQV RV32EC (Michael Bell, Apache-2.0), 64 MHz, code runs from QSPI flash
Author	Santhosh (GitHub `SANTHOSH2730`)
Status	Hardened and signed off, Tiny Tapeout precheck passed, layout reproduced bit for bit by a clean re-run, 11/11 RTL and gate-level tests pass, Vivado FPGA build and FPGA simulations pass (board test pending)
Guide date	10 November 2026

How to use this guide. Sections 1-2 are the one-page overview. Section 3 explains, test by test, what the chip actually did in simulation. Sections 4-6 are the three ways to run it (laptop simulation, PYNQ-Z2 FPGA with Vivado, the real chip). Sections 7-8 are for writing your own programs and applications. Sections 9-11 are limits, troubleshooting and a map of the files. Every number here comes from a log or report in this repository; the source is named next to it.

Related documents (all in `docs/`, PDFs in `docs/pdf/`): `info.md` (datasheet), `IO_GUIDE.md` (pin by pin), `fpga.md` (FPGA quick start), `DESIGN_REPORT.md` (full design and implementation report).

---

1. The chip in one page



The CPU has no memory inside the chip. It fetches its program over a 4-bit QSPI bus from an external flash chip (execute in place) and keeps variables and the stack in external PSRAM on the same bus. Everything else (UART, GPIO, timer, interrupts) is on the chip.

Item	Value	Source
Instruction set	RV32E + C + Zicsr, `ecall`, `ebreak`, `mret`	`docs/info.md`
Clock	64 MHz nominal (typical-corner limit about 71 MHz)	STA, `reports/summary.md`
Program memory	QSPI flash, up to 16 MB, at 0x0000000	`firmware/link.ld`
Data memory	PSRAM A 8 MB at 0x1000000, PSRAM B 8 MB at 0x1800000	`docs/info.md`
Peripherals	UART (TX, RX, RTS), 8 GPIO in, 8 GPIO out, timer + compare	`firmware/rvsoc.h`
Interrupts	ui[0] edge, ui[1] edge, UART RX, timer	`docs/info.md`

Item	Value	Source
Die area used	2 x 2 tiles, 5,752 standard cells, 78.4 % utilisation	`reports/sweep.md`
Power (estimate)	6.6 mW at 64 MHz, typical corner	`reports/summary.md`

2. Pins at a glance

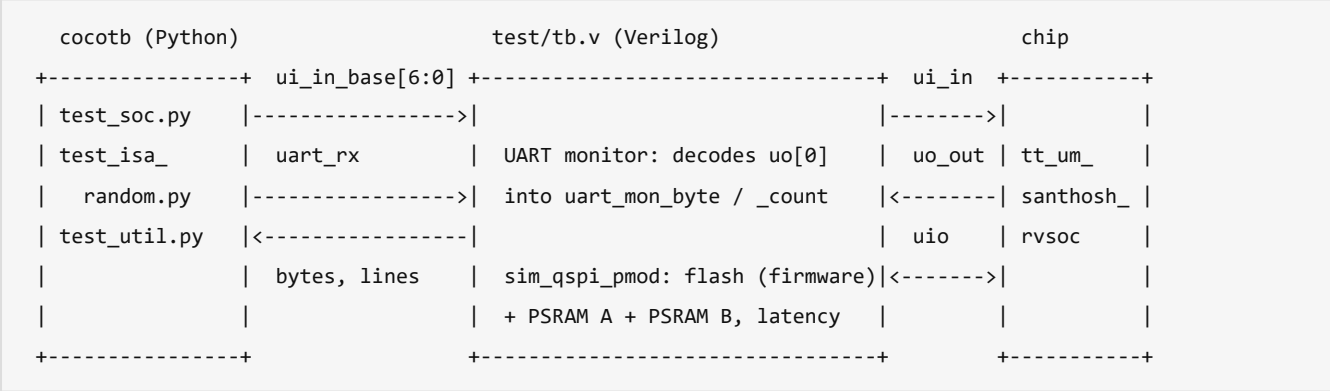
Pin	Function		Pin	Function
`clk`	64 MHz clock		`rst_n`	reset, active low
ui[0]	interrupt 0 / GPIO in 0		uo[0]	UART TX (or GPIO out 0)
ui[1]	interrupt 1 / GPIO in 1		uo[1]	UART RTS (or GPIO out 1)
ui[2], ui[3]	mode switches (read at boot) / GPIO		uo[2]..uo[7]	GPIO out (demo: LED chaser)
ui[4]	**keep low** (simulation speed-up)		uio[0]	flash CS#
ui[5], ui[6]	GPIO in		uio[1], [2], [4], [5]	QSPI SD0, SD1, SD2, SD3
ui[7]	UART RX		uio[3]	QSPI SCK (32 MHz)
			uio[6], uio[7]	PSRAM A CS#, PSRAM B CS#

During reset the chip reads its **QSPI read latency** (0-5) from SD2:SD0. The full pin-by-pin description, including the PYNQ-Z2 mapping, is `docs/IO_GUIDE.md`.

---

3. How the chip behaves: what the simulations show

3.1 How the testbench sees the chip



- The **same testbench runs on two designs**: the RTL (`src/`) and the post-route gate-level netlist of the real layout (`runs/wokwi/final/pnl/`, with power pins).
- Two kinds of tests:
  1. **System tests** (`test/test_soc.py`, 7 tests). The real firmware (`firmware/`) is loaded into the flash model and runs. The tests only look at the chip's pins: UART text, LED pins, PSRAM contents.
  2. **Instruction tests** (`test/test_isa_random.py`, 4 tests, ported from upstream TinyQV). Python plays the flash: it feeds instructions over the QSPI bus one by one and reads registers back by catching stores on the bus.
- Most system tests set ui[4] = 1 (FAST mode: the firmware switches the UART to 2 Mbaud and the timer tick to 250 us) so that the simulation finishes in minutes. `test_boot_hello` also boots once at the real 115200 baud.

3.2 Reset and boot, as seen on the pins

Step	What the testbench does	What the chip does	Checked
1	`rst_n` = 0, SD2:SD0 = latency (1, 2 or 3)	all `uio` pins are inputs	`uio_oe == 0`
2	10 clocks later, `rst_n` = 1	CS#, SCK become outputs	`uio_oe == 0b11001001`
3	-	flash CS# low, command-less XIP read from address 0	flash model
4	-	`start.S`: stack in PSRAM A, copy `.data`, clear `.bss`, call `main`	PSRAM model
5	-	`main`: read GPIO_IN (mode, FAST), set UART speed, print banner	UART monitor

3.3 Test by test (logs: `logs/rtl_test.log`, `logs/gl_test.log`)

All 11 tests pass on the RTL **and** on the gate-level netlist. The UART lines and their simulated times are **identical** in both (same nanosecond), which means the real layout behaves cycle for cycle like the RTL.

Test 1: `test_boot_hello`: does the chip boot at all QSPI latencies?

- *Stimulus:* three resets: latency 1 at real speed (115200 baud), then latency 2 and 3 in FAST mode.
- *Output seen:*

Boot	UART output	Time from reset to end of banner
latency 1, 115200 baud	`Hello, TT!`	1,117 us
latency 2, FAST	`Hello, TT!`	132 us
latency 3, FAST	`Hello, TT!`	132 us

- *Why 1,117 us:* the banner is 12 characters x 86.9 us = 1,043 us of serial time, plus about 75 us of start-up code. At 2 Mbaud the same banner takes 60 us.
- *Proves:* the CPU starts from flash, the QSPI controller works with read latency 1, 2 and 3, the UART divider reset value (555 = 115200 baud) is right, and there are no UART framing errors.

Test 2: `test_isa_selftest`: does every instruction work?

- *Stimulus:* boot in FAST mode.
- *Output seen:* `Hello, TT!`, then `ISA PASS` 430 us later.
- *What ran:* `firmware/isa_test.S`, about 90 numbered checks: all register and immediate ALU operations (1-19), LUI/AUIPC (20-22), all branch types taken and not taken (23-36), JAL/JALR (37-39), byte/half/word loads and stores to PSRAM, including negative offsets and data reads from flash (40-47), FENCE, x0, the fixed gp/tp registers (48-50), CSR instructions on `mepc`, `mie`, `mstatus`, `misa` (51-55), `ecall`/`ebreak` traps with `mcause`/`mepc`/`mret` (56-57) and the compressed (C) instructions (70 onwards). A failure would print `ISA FAIL n` with the check number.
- *Proves:* the whole ISA executes correctly from real QSPI flash, including the two upstream bugs fixed in this design (FENCE and the CSR-immediate forms).

Test 3: `test_psram`: do both data memories work?

- *Output seen:* `ISA PASS`, then `PSRAM PASS` 346 us later.

- *Extra check by the testbench:* after the firmware passes, Python reads the PSRAM models directly: 16 words at PSRAM A 0x8000 (0x12345678, then +0x11111111 each), 16 words at PSRAM B 0x0100 (0xA5A55A5A, XOR/add pattern) and the bytes C0 C1 C2 C3 written one at a time at PSRAM A 0x8040, read back as the word 0xC3C2C1C0.
- *Proves:* the data really went out over the QSPI bus to the right chip at the right address, with byte, halfword and word sizes (the firmware could not fake this).

Test 4: `test_timer_irq`: do the timer and its interrupt work?

- *Output seen (FAST mode, times after the banner):*

Line	Time after previous line	What happened before it
`ISA PASS`	430 us	instruction self-test
`PSRAM PASS`	346 us	PSRAM test
`TIMER PASS`	1,006 us	counter runs, is writable, 3 compare interrupts with `mcause` 0x80000013
`GPIO PASS`	119 us	GPIO_OUT/GPIO_SEL read-back
`SELFTEST PASS`	118 us	-
`tick 10`	2,695 us	10 timer interrupts of 250 us, LED chaser moving

- *LED check:* the testbench watches uo[7:2] and requires exactly one LED lit at a time and 5 steps whose spacing is within 15,500-16,500 clocks (the firmware sets 2,000 timer ticks x 8 clocks = 16,000 clocks = 250 us). All steps passed.

- *Proves:* timer counting, compare match, interrupt entry/exit (`irq_entry` in `start.S`, `mret`), interrupt acknowledge by writing TIMER_CMP, and that the interrupt timing is exact to the clock.

Test 5: `test_gpio_loopback`: do the GPIO pins work both ways?

- *Stimulus:* boot in mirror mode (ui[3:2] = 01), then 12 random 8-bit values on ui[7:0], each held for 400 clocks (6.25 us).
- *Output seen:* `Hello, TT!`, `GPIO PASS`, `MIRROR`; then for every value uo[7:2] = ui[7:2], and uo[0] (UART TX) stayed high.
- *Proves:* every GPIO input and output bit, and that the firmware's GPIO loop reacts within a few microseconds.

Test 6: `test_uart_echo`: does the UART receiver work?

- *Stimulus:* after `SELFTEST PASS`, Python sends `H`, `i`, space, `T`, `T`, Enter on ui[7] (8N1, 2 Mbaud), waiting for each echo before the next character.
- *Output seen:* `Hi TT` followed by CR LF (the firmware turns Enter into CR LF).
- *Proves:* UART RX, the RX-valid flag, reading UART_DATA, and that RX and TX work together.

Test 7: `test_ext_irq`: do the external interrupts work?

- *Stimulus:* a 200-clock high pulse on ui[0], then on ui[1].
- *Output seen:* `IRQ0`, then `IRQ1` (136 us apart).
- *Proves:* rising-edge detection on both pins, the interrupt enable bits in `mie`, and clearing the edge interrupt in `mip`.

Test 8: `test_start` (upstream): bus-level UART and GPIO

- Python feeds instructions directly. It checks the UART TX waveform bit by bit (byte 0x54 at 8.68 us per bit), sends 10 random bytes into UART RX while checking the RTS pin goes high when a byte is waiting and low when it has been read, and writes 40 random GPIO_SEL/GPIO_OUT combinations checking uo[7:0].

Test 9: `test_load_bug` (upstream): a load corner case

- Compressed `c.lwsp` loads based on `tp` (peripheral space: reads the GPIO input value 0b01101000) and on `sp` (PSRAM), back to back. This is a regression test for a load bug found in the original TinyQV.

Test 10: `test_random_alu` (upstream): random ALU programs

- 15 registers set to random values, then 200 random ALU instructions; all 16 registers are read back and compared with a Python model of RISC-V.
- RTL: 25 seeds (5,000 instructions). Gate level: 3 seeds (600 instructions; the gate-level simulator is about 7 times slower).

Test 11: `test_random` (upstream): random programs with memory

- As test 10, but 1,000 random instructions per seed including loads and stores to random PSRAM addresses.
- RTL: 5 seeds (5,000 instructions). Gate level: 1 seed (1,000 instructions).

3.4 RTL against the real layout

Test	Simulated time, RTL	Simulated time, gate level	Same?
test_boot_hello	1,381,630 ns	1,381,630 ns	identical
test_isa_selftest	561,698 ns	561,698 ns	identical
test_psram	907,426 ns	907,426 ns	identical
test_timer_irq	4,845,674 ns	4,845,674 ns	identical
test_gpio_loopback	393,584 ns	393,584 ns	identical
test_uart_echo	2,375,332 ns	2,375,332 ns	identical
test_ext_irq	2,482,450 ns	2,482,450 ns	identical
test_start	1,078,079 ns	1,078,079 ns	identical
test_load_bug	6,007 ns	6,007 ns	identical
test_random_alu	2,075,578 ns (25 seeds)	246,570 ns (3 seeds)	fewer seeds on gates
test_random	3,380,370 ns (5 seeds)	677,980 ns (1 seed)	fewer seeds on gates
Total	11/11 pass, 44 min	11/11 pass, 56 min	

For the 9 tests that do not use random seeds, the gate-level run ended at exactly the same nanosecond as the RTL run, with every UART line at the same time. The synthesis, placement, clock tree and routing changed nothing in the behaviour.

3.5 Looking at the waveforms yourself

The RTL run writes `test/tb.fst` (pins only, so it stays small). Open it in GTKWave with the prepared signal list:

```
cd ~/tt-sky26d-rvsoc/test && gtkwave tb.fst tb.gtkw
```

What to look at: `rst_n` rising, then `uio_out[0]` (flash CS#) going low and `uio_out[3]` (SCK) toggling at 32 MHz: that is the CPU fetching code. `uio_out[6]` low pulses are PSRAM A accesses (stack and variables). `uo_out[0]` is the UART text; `uo_out[7:2]` shows the LED chaser. For more internal detail, re-run one test with `make COCOTB_TEST_MODULES=test_soc COCOTB_TEST_FILTER=test_timer_irq EXTRA_DEFINES="-DTRACE_FETCH -DTRACE_TIMER"`: the log then prints every flash fetch address and every timer access.

---

4. Running it on your laptop (simulation)

Everything is installed in WSL (`~/ttsetup`). From a WSL terminal:

```
cd ~/tt-sky26d-rvsoc && source scripts/env.sh
```

Task	Command	Time
Check the tools	`scripts/check_env.sh`	1 min
Build the firmware	`make -C firmware`	seconds
All RTL tests	`make -C test`	44 min
One test	`make -C test COCOTB_TEST_MODULES=test_soc COCOTB_TEST_FILTER=test_uart_echo`	2-5 min
Gate-level tests	`scripts/gl_test.sh`	56 min
FPGA harness (Icarus)	`make -C fpga sim`	2 min
Everything (incl. 5 h hardening)	`scripts/run_all.sh`	about 7 h

After changing the firmware, copy the new image into the testbench folder: `cp firmware/build/rvsoc_fw.hex test/` (scripts/run_all.sh does this for you).

---

5. Running it on the PYNQ-Z2 with Vivado

The FPGA version uses the **unchanged chip RTL** plus a small harness. The QSPI flash and both PSRAMs are emulated in FPGA block RAM (64 KB each), with the firmware built into the bitstream, so no Pmod memory board is needed.

5.1 One command does the whole Vivado flow

Open Windows PowerShell and run:

```
powershell -ExecutionPolicy Bypass -File \\wsl.localhost\Ubuntu-22.04\home\santhosh\tt-sky26d-rvsoc\fpga\pynq_z2\build_on_windows.ps1
```

Step	What it does	Output
1	builds the firmware in WSL and converts it to a block-RAM image	`fpga/build/rvsoc_fw_nibbles.hex`
2	**xsim, RTL, fast**: `fpga/sim/tb_fpga.v` (MMCM bypassed, FAST mode)	`reports/fpga/xsim_rtl_fast.log`

Step	What it does	Output
3	**xsim, RTL, board**: `fpga/sim/tb_fpga_board.v`, real MMCM model, 115200 baud, pins only	`reports/fpga/xsim_rtl_board.log`
4	**Vivado** synthesis, placement, routing, reports, bitstream	`fpga/build/rvsoc_pynq_z2.bit`, `reports/fpga/*.rpt`
5	**xsim, netlist, board**: the same board test on Vivado's post-route netlist, i.e. the logic inside the bitstream	`reports/fpga/xsim_netlist_board.log`
6	only with `-Program`: loads the bitstream over JTAG	-

Options: `-SkipFirmware`, `-SkipSim`, `-SkipBuild`, `-Program`. The script copies the sources to `%TEMP%\rvsoc_vivado` first, because Vivado is much faster on a local disk than on the `\\wsl.localhost` share. The summary is written to `reports/fpga/vivado_summary.txt`.

5.2 Results of the Vivado flow on this laptop

Run on 10 November 2026, Vivado 2025.1, part xc7z020clg400-1. Logs and reports are in `reports/fpga/`.

Step	Result	Run time
xsim, RTL, fast (`tb_fpga.v`)	**PASS**: `Hello, TT!`, ISA/PSRAM/TIMER/GPIO PASS, `SELFTEST PASS`; the on-FPGA checker set "hello" and "self-test" and saw no FAIL; LD0-LD3 moved 9 times. Same result as Icarus (`make -C fpga sim`)	4.3 min
xsim, RTL, board (`tb_fpga_board.v`)	**PASS**: all 12 pin-level checks (list below), 119 UART characters, 12.8 ms simulated	13.8 min
Synthesis	0 errors, 0 critical warnings, 16 harmless warnings (explained below)	9 min
Placement and routing	**timing met at 64 MHz**: setup WNS +2.126 ns, hold WHS +0.052 ns, 0 failing of 2,659 endpoints	6 min
Bitstream	`fpga/build/rvsoc_pynq_z2.bit`, 4.0 MB	1 min
xsim, post-route netlist, board	**PASS**: the same 12 checks on the logic inside the bitstream; identical UART text and identical times (banner 1,134 us, self-test 7,801 us)	50 min

What the board-level test checks (`fpga/sim/tb_fpga_board.v`; it drives and reads only the board pins, with the real 125 MHz clock, the MMCM and 115200 baud):

#	Check	Seen
1	BTN0 released (SW = 00) -> `Hello, TT!`	banner complete **1,134 us** after release
2-6	`ISA PASS`, `PSRAM PASS`, `TIMER PASS`, `GPIO PASS`, `SELFTEST PASS`	self-test complete **7,801 us** after release
7	LD4 green and LD5 green lit, LD5 never red	ok
8	flash CS# (Pmod JA7) and PSRAM A CS# (JA10) toggle	ok
9	BTN1 -> `IRQ0`, BTN2 -> `IRQ1`	ok
10	typing `H`, `i`, Enter into Pmod JA2 -> echo `Hi` + CR LF	ok
11	BTN0 with SW = 01 -> `Hello, TT!`, `GPIO PASS`, `MIRROR`	ok
12	mirror mode: LD0/LD1/LD3 and Pmod JB follow SW0, SW1, BTN3 and JA4, also when released; UART TX idles high; no framing errors	ok

The banner time matches the chip-level cocotb test (1,117 us): the FPGA harness holds reset 1,024 clocks (16 us) longer after BTN0 and uses read latency 2.

Resources (`reports/fpga/utilization.rpt`, `utilization_hier.rpt`):

Resource	Used	Available	%
LUTs	2,035	53,200	3.8
Flip-flops	952	106,400	0.9
Block RAM (36 Kb)	32	140	22.9
I/O pins	33	125	26.4
MMCM	1	4	25

Module	LUTs	of which shift registers	Flip-flops	Block RAM
`i_chip`: the chip (`tt_um_santhosh_rvsoc`)	1,155	48	661	0
- TinyQV CPU core	967	48	396	0
- QSPI memory controller	128	0	103	0
- timer	8	0	67	0
`i_mem`: flash + 2 PSRAM emulator	803	0	185	32
`i_watch`: UART checker for the LEDs	68	0	53	0
top level: input sync, reset, LEDs	11	0	53	0

- The chip itself needs only about 1,150 LUTs. On the FPGA the CPU's rotating register file becomes 48 shift-register LUTs, so far fewer flip-flops are needed than the 986 on silicon.
- Each PSRAM uses 16 block RAMs (128 K nibbles = 64 KB). The flash is a ROM: Vivado trims it to the part the firmware fills (16 K nibbles for the 4.6 KB demo) and builds it from about 800 LUTs. A bigger program simply uses more LUTs (a full 64 KB image would still fit easily).
- **Warnings, all expected:** two RGB LED pins are tied to 0 on purpose (LD4 red, LD5 blue); `ena` and `uio_in[0,3,6,7]` are unused (those QSPI pins are outputs); the register file has no reset (as on the chip); the design is too small for parallel synthesis. DRC: one warning, ZPS7-1 "PS7 block required": the design uses only the FPGA fabric, not the ARM side, which is fine for JTAG loading. Methodology: 16 notes that the PSRAM block RAMs have no extra output register (timing is met anyway).
- **Power** (Vivado estimate, low confidence because no switching activity was given): 0.248 W, of which 0.108 W is the MMCM and 0.110 W the FPGA's static power. This says nothing about the chip, whose estimate is 6.6 mW.

5.3 Doing the same in the Vivado GUI (optional)

- **Look at the result:** start Vivado, *File* > *Checkpoint* > *Open*, `fpga\build\top_pynq_z2_routed.dcp`. *Window* > *Device* shows the placed design, *Reports* > *Timing* > *Report Timing Summary* the timing.
- **Make a GUI project:** *Create Project* (RTL project, part `xc7z020clg400-1`); add the files from `src/tinyQV/cpu/`, `src/tinyQV/peri/uart/`, `src/timer.v`, `src/project.v` and set their *File Type* to **SystemVerilog**; add `fpga/rtl/*.v` and `fpga/pynq_z2/top_pynq_z2.v`; add the constraint file `fpga/pynq_z2/pynq_z2.xdc`; set the top to `top_pynq_z2`; in *Settings* > *Synthesis* > *More options* set `-generic FLASH_INIT="<full path to rvsoc_fw_nibbles.hex>"`. Then *Generate Bitstream*.
- **Simulation with waveforms:** add `fpga/sim/tb_fpga_board.v` as a simulation source, set it as the simulation top and run *Behavioral Simulation*. Run for 15 ms (the board test finishes by itself and prints `[tb_board] PASS`).

5.4 Loading the bitstream and what you will see

1. Connect the PYNQ-Z2 micro-USB (PROG/UART) port to the laptop and switch the board on. Either set the boot jumper to JTAG, or keep SD: JTAG programming works in both cases.
2. Program it: run the script with `-SkipSim -SkipBuild -Program` (loads the existing bitstream in about a minute), or in Vivado *Open Hardware Manager* > *Open target* > *Auto connect* > *Program device* and pick `fpga\build\rvsoc_pynq_z2.bit`. With the PYNQ Linux image you can also copy the file to the board and run, in Jupyter: `from pynq import Bitstream; Bitstream("rvsoc_pynq_z2.bit").download()`.
3. **No PC terminal needed for a first check:** LD4 blinks blue (FPGA alive). Press and release **BTN0**. Within about 10 ms **LD4 turns green** (`Hello, TT!` was printed) and **LD5 turns green** (`SELFTEST PASS`). LD5 red would mean a `FAIL` line. Then LD0-LD3 light one after another, one step every 100 ms.
4. **Full console:** connect a 3.3 V USB-UART adapter (CP2102, CH340 or FT232) to Pmod JA: adapter **RX to JA pin 1**, adapter **TX to JA pin 2**, **GND to JA pin 5**. Open the adapter's COM port (Device Manager shows the number) in PuTTY or Tera Term at **115200, 8 data bits, no parity, 1 stop bit, no flow control**. Press BTN0: you see the same text as on the real chip (section 6.4). Press BTN1 for `IRQ0`, BTN2 for `IRQ1`; type text and it is echoed.
5. **Mirror mode:** set SW0 = 1, SW1 = 0, press BTN0. LD0 follows SW0, LD1 follows SW1, LD3 follows BTN3, Pmod JB pin 9 follows Pmod JA pin 4.
6. **Logic analyser:** Pmod JA pins 7-10 carry flash CS#, SCK, SD0 and PSRAM A CS#; Pmod JB carries all eight chip outputs uo[0..7].

The PYNQ-Z2's own USB-UART is wired to the ARM processor, not to the FPGA fabric; that is why the chip's UART goes to Pmod JA. The board's ARM cores are not used.

---

6. Running it on the real chip (Tiny Tapeout demo board)

6.1 What you need

- The Tiny Tapeout SKY 26d chip on its demo board (RP2040 controller), with this design selected.
- A **QSPI Pmod** with a 16 MB flash and two 8 MB PSRAMs (mole99/qspi-pmod, the board used by TinyQV on TT06) plugged into the **bidirectional (uio) Pmod**.
- A serial terminal on the chip's UART (uo[0] TX, ui[7] RX), either through the demo board's RP2040 or a 3.3 V USB-UART adapter on the Pmod headers.

6.2 Step by step

1. **Build the firmware:** `make -C firmware` gives `firmware/build/rvsoc_fw.bin` (4.6 KB).
2. **Program the flash:** write `rvsoc_fw.bin` to flash offset 0 of the QSPI Pmod. The Pmod, pinout and boot protocol are the same as TT06 TinyQV, so the TinyQV flash-programming procedure for the demo board applies unchanged.
3. **Select the design and set the clock** to 64 MHz with the demo board software. A lower clock also works; the UART baud rate then scales with it (section 7.5).
4. **Reset sequence** (the RP2040 drives the uio pins while the chip is in reset):
    - drive `rst_n` low;
    - put the flash in continuous-read (XIP) mode and both PSRAMs in QPI mode;
    - drive all three CS# high and SD2:SD0 = read latency (**start with 2**; try 3 if the chip does not boot);

    - give at least 8 clocks, release the QSPI lines (high impedance), then raise `rst_n` while `clk` is high.
5. Keep ui[2]..ui[6] low (ui[4] must be low on real hardware).
6. Open the terminal at 115200 8N1. Expected output within about 8 ms of reset (7.8 ms in the board-level simulation, most of it serial-line time):

```
Hello, TT!
ISA PASS
PSRAM PASS
TIMER PASS
GPIO PASS
SELFTEST PASS
tick 10
tick 20
...
```

`tick N` lines come once a second. LEDs on uo[2..7] show a light running back and forth (100 ms per step). Pulses on ui[0]/ui[1] print `IRQ0` / `IRQ1`. Characters you type are echoed.

6.3 Choosing the read latency

The read latency is the number of extra clocks the chip waits for data after the QSPI dummy cycles. It has to cover the delay through the chip's I/O pads, the board and the memory chip. In simulation latency 1, 2 and 3 all work; on the FPGA it is 2. On the real chip use 2 at 64 MHz and 3 if you see garbage or no output. If neither works, lower the clock (for example 32 MHz) to check that the rest of the setup is right.

6.4 The UART text explained

Line	Meaning	If it fails
`Hello, TT!`	CPU runs from flash, UART works	no text: check flash content, XIP mode, latency, clock, TX wiring
`ISA PASS`	all instructions correct	`ISA FAIL n`: check number n in `firmware/isa_test.S`
`PSRAM PASS`	both PSRAMs read and write	`PSRAM FAIL n`: 1-16 PSRAM A word, 17-32 PSRAM B word, 33-48 byte/halfword, 49 byte order
`TIMER PASS`	timer and interrupt	1 not counting, 2 not writable, 3 no interrupt, 4 wrong `mcause`, 5 extra interrupts
`GPIO PASS`	output register read-back	1-7: which GPIO_OUT/GPIO_SEL step failed
`SELFTEST PASS` / `FAIL`	summary	-
`tick N`	timer interrupt count, every 10 ticks	-
`IRQ0`, `IRQ1`	rising edge on ui[0], ui[1]	-
`MIRROR`	mirror mode entered (ui[3:2] = 01)	-

7. Writing your own programs

7.1 Tools and build

The toolchain is the xPack `riscv-none-elf-gcc` 14.3 in WSL (put on the PATH by `scripts/env.sh`). Compiler flags that matter (from `firmware/Makefile`):

Flag	Why
`-march=rv32ec_zicsr -mabi=ilp32e`	16 registers, compressed instructions, CSR access
`-Os`	code runs from flash over QSPI: smaller code is faster code
`-msmall-data-limit=0 -mno-relax`	`gp` is hard-wired to 0x1000400 in TinyQV; the linker must not use it for relaxation
`-ffreestanding -nostdlib -nostartfiles`	no C library; `start.S` and `link.ld` set things up

Start from the existing firmware: copy `firmware/` to a new folder, keep `start.S`, `link.ld`, `rvsoc.h`, and replace `main.c` (and drop `isa_test.S` from `SRCS` if you do not need it).

7.2 Memory layout rules

Region	Address	Used for	Notes
Flash	0x0000000-0x0FFFFFF	code, constants, initial values of `.data`	read only; fetching is serial, so tight loops are slower than on a normal MCU
PSRAM A	0x1000000-0x17FFFFF	`.data`, `.bss`, stack (top of the first 64 KB)	`link.ld` uses 64 KB; enlarge `LENGTH` up to 8 MB if needed
PSRAM B	0x1800000-0x1FFFFFF	free	use it for large buffers via a pointer
Peripherals	0x8000000-0x800003F	registers	always use 32-bit `volatile` accesses

Fixed registers: `gp` (x3) always reads 0x1000400 and `tp` (x4) always reads 0x8000000. The compiler is told not to use them; do not write to them in assembly.

7.3 Register reference card

```
#define GPIO_OUT     (*(volatile uint32_t *)0x8000000)  // R/W  uo[7:0]; read returns the pins
#define GPIO_IN      (*(volatile uint32_t *)0x8000004)  // R    ui[7:0]
#define GPIO_SEL     (*(volatile uint32_t *)0x800000C)  // R/W  bit0: uo[0] from GPIO_OUT, bit1: uo[1]
from GPIO_OUT
#define UART_DATA    (*(volatile uint32_t *)0x8000010)  // W: send byte, R: received byte (clears RX
valid)
#define UART_STATUS  (*(volatile uint32_t *)0x8000014)  // R    bit0 TX busy, bit1 RX byte valid
#define TIMER_COUNT  (*(volatile uint32_t *)0x8000028)  // R/W  +1 every 8 clocks (8 MHz at 64 MHz)
#define TIMER_CMP    (*(volatile uint32_t *)0x800002C)  // R/W  interrupt when COUNT == CMP; write clears
it
#define UART_DIV     (*(volatile uint32_t *)0x8000034)  // R/W  clocks per bit - 1 (13 bits), reset 555
```

Interrupt	`mcause`	Enable bit in `mie`	Type	How to clear
ui[0] rising edge	0x80000010	16	edge	`csr_clear(mip, 1 << 16)`

Interrupt	`mcause`	Enable bit in `mie`	Type	How to clear
ui[1] rising edge	0x80000011	17	edge	`csr_clear(mip, 1 << 17)`
UART byte received	0x80000012	18	level	read UART_DATA
Timer compare	0x80000013	19	level	write TIMER_CMP

`mstatus.MIE` is 1 after reset, so setting a bit in `mie` is enough to enable an interrupt. Vectors: reset at 0x0, `ecall` / `ebreak` at 0x4, interrupts at 0x8; `start.S` jumps from there to `trap_handler()` and `irq_handler()` in C.

7.4 Example code

All snippets use `rvsoc.h` and compile with the firmware Makefile flags (checked with the toolchain).

Wait a number of microseconds (timer, no interrupt):

```
static void delay_us(uint32_t us)
{
    uint32_t t0 = TIMER_COUNT;
    while (TIMER_COUNT - t0 < us * 8u) { }       /* 8 ticks per us at 64 MHz */
}
```

Blink an LED on uo[2]:

```
int main(void)
{
    for (;;) {
        GPIO_OUT ^= 0x04;                        /* toggle uo[2] */
        delay_us(500000);                        /* 0.5 s */
    }
}
```

Print text and numbers, read characters (polling):

```
static void putc_(char c) { while (UART_STATUS & UART_STATUS_TX_BUSY) { } UART_DATA = (uint8_t)c; }
static void puts_(const char *s) { while (*s) putc_(*s++); }
static int  getc_nonblock(void) { return (UART_STATUS & UART_STATUS_RX_VALID) ? (int)(UART_DATA & 0xFF) :
-1; }
```

(`main.c` also has `uart_putdec()` for decimal numbers; there is no hardware multiply or divide, so it uses repeated subtraction.)

Periodic timer interrupt (here 1 kHz):

```
#define PERIOD 8000u                            /* 8000 ticks = 1 ms at 64 MHz */
volatile uint32_t ms;

void irq_handler(uint32_t cause)
{
    if ((cause & 0x1fu) == IRQ_TIMER) {
        TIMER_CMP = TIMER_CMP + PERIOD;         /* next match; also clears the interrupt */
        ms++;
    }
}

int main(void)
{
    TIMER_CMP = TIMER_COUNT + PERIOD;
    csr_set(mie, 1u << IRQ_TIMER);
    for (;;) { /* ms counts up in the background */ }
}
```

Keep the period well above the interrupt cost: a C handler that saves 10 registers takes about 2,400 clocks (37 us at 64 MHz), because code is fetched from flash and the stack is in PSRAM. 1 kHz uses about 4 % of the CPU; 10 kHz would use 37 %.

Button interrupt on ui[0]:

```
volatile uint32_t presses;

void irq_handler(uint32_t cause)
{
    if ((cause & 0x1fu) == IRQ_IN0) {
        csr_clear(mip, 1u << IRQ_IN0);          /* edge interrupts are cleared in mip */
        presses++;
    }
}

int main(void)
{
    csr_clear(mip, 1u << IRQ_IN0);              /* forget an edge seen before */
    csr_set(mie, 1u << IRQ_IN0);
    for (;;) { }
}
```

A mechanical button bounces: count presses in the main loop with a short delay, or ignore edges that come less than about 10 ms after the previous one (compare TIMER_COUNT).

Change the baud rate: `UART_DIV = f_clk / baud - 1`. At 64 MHz: 555 = 115200, 277 = 230400, 63 = 1 Mbaud, 31 = 2 Mbaud. Wait until TX is idle before changing it.

Use uo[0] and uo[1] as plain outputs: set the GPIO_OUT bits first, then `GPIO_SEL = 3`. Set bit 0 high before switching if a terminal is connected, so the TX line does not drop low (a receiver would see a false

start bit).

Big buffer in PSRAM B:

```
static volatile uint8_t *const bufB = (volatile uint8_t *)0x1800000;   /* 8 MB */
```

7.5 Running your program

Where	How
Simulation	`make -C firmware && cp firmware/build/rvsoc_fw.hex test/`, then write a cocotb test like `test_soc.py` (use `boot_firmware()` and `mon.expect_line()` from `test_util.py`)
FPGA	run `build_on_windows.ps1` again: it rebuilds the firmware and puts it into the block-RAM flash (programs up to 64 KB)
Real chip	program the new `rvsoc_fw.bin` into the QSPI flash at offset 0 and reset

If you run the chip at a clock other than 64 MHz, set `UART_DIV` and timer periods from the real clock: `UART_DIV = f_clk / 115200 - 1`, timer tick = 8 / f_clk.

7.6 Speed you can expect

Operation	Approximate time at 64 MHz	Source
Simple instruction	8 clocks (32 bits processed 4 per clock), plus flash fetch	TinyQV design
Interrupt with a C handler	2,400 clocks = 37 us	`firmware/main.c`
GPIO input -> output in a loop	about 1 us	mirror mode, `test_gpio_loopback`
UART character at 115200	87 us	556 clocks per bit
Boot to banner at 115200	1.1 ms	`test_boot_hello`

This is a small, slow-but-complete 32-bit CPU: roughly the speed of a classic 8-bit microcontroller, with a full 32-bit instruction set and megabytes of memory.

---

8. Application ideas

Each idea fits the pins and speed of this chip; code is a few hundred lines of C on top of the examples above.

#	Application	Pins	How it works	Limit
1	**UART command shell / I/O expander**	UART, ui, uo	Parse text commands (`set 3 1`, `get`, `pulse 4 100`) and drive or read the pins; a PC script controls real hardware through it	about 1,000 commands/s at 115200
2	**Frequency and event counter**	ui[0] (signal), UART	Count ui[0] edge interrupts during a 1 s timer gate, print the count	about 20 kHz (37 us per interrupt)
3	**Stopwatch / reaction-time game**	BTN on ui[0], ui[1], LEDs on uo	Light an LED after a random delay, measure the button press with TIMER_COUNT (125 ns resolution), print the result	-
4	**Software PWM, LED dimmer, servo**	uo[2..7]	Timer interrupt at 1-5 kHz updates the outputs from duty-cycle tables; for a servo, 50 Hz frames with 1-2 ms pulses	1 % steps at 100 Hz PWM
5	**Data logger**	ui inputs, UART	Sample ui[7:0] at a fixed rate into the 8 MB PSRAM B, dump it as hex over UART on request	about 10 k samples/s

#	Application	Pins	How it works	Limit
6	**Bit-banged I2C or SPI master**	uo for SCL/SCK/MOSI, ui for SDA-in/MISO	Talk to sensors or displays in software (an I2C SDA line needs an external open-drain buffer, since uo pins are push-pull)	tens of kHz bit rate
7	**RISC-V teaching board**	UART	Students write C or assembly, see results on the terminal; `isa_test.S` is a worked reference for every instruction	-
8	**Morse code / UART-to-light beacon**	UART in, uo out	Text typed on the terminal is sent as Morse on an LED or buzzer pin	-

Typical hardware around the chip: the QSPI Pmod (always), a USB-UART, a few buttons on ui[0..1] with pull-downs, LEDs with series resistors on uo[2..7].

---

9. Operating limits

Item	Value	Note
Clock	64 MHz design point	Setup slack +1.59 ns at the typical corner (limit about 71 MHz); the slow corner needs about 36 MHz, so a slow part may need a lower clock
Reset	`rst_n` low for at least 8 clocks	raise it while `clk` is high
QSPI read latency	1-3 in simulation, 2 on the FPGA	set on SD2:SD0 during reset (section 6.3)
UART	115200 8N1 at reset; `UART_DIV` 13 bits	1-byte receive buffer: read within one character time (87 us at 115200)
Timer	8 MHz at 64 MHz, 32 bits	wraps after 537 s; compare interrupt needs a period well above 37 us
ui[4]	must be low on hardware	high selects the simulation speed-up
Power	about 6.6 mW at 64 MHz (STA estimate)	`reports/summary.md`
Missing on purpose	hardware multiply/divide, cycle counters, misaligned-access and illegal-instruction traps	removed or never present to fit 2x2 tiles (`docs/info.md`)

---

10. Troubleshooting

Symptom	Likely cause	What to do
No UART text at all (chip)	flash empty or not in XIP mode, wrong latency, clock off, TX wire	check flash content, try latency 2 then 3, lower the clock, check TX goes high after reset
Garbage characters	baud mismatch (clock not 64 MHz, or ui[4] high)	set the terminal to `f_clk / 556` baud, keep ui[4] low
`Hello, TT!` then nothing	PSRAM not in QPI mode or wrong CS# wiring (stack is in PSRAM)	check the PSRAM setup in the reset sequence
`PSRAM FAIL n`	data path or PSRAM problem	n tells which word or size (section 6.4)
`TIMER FAIL 3`	interrupt never fired	`mie` bit 19 not set, or TIMER_CMP set in the past (it fires only on equality)
Echo loses characters	1-byte receive buffer	send one character at a time, or use the UART interrupt
IRQ printed twice for one press	button bounce	debounce (section 7.4)

Symptom	Likely cause	What to do
FPGA: LD4 never blinks blue	bitstream not loaded	program again, check the JTAG cable and power
FPGA: LD4 blue but not green after BTN0	firmware image not in the bitstream	rebuild with `build_on_windows.ps1` (it rebuilds the firmware)
FPGA: LD5 red	a self-test failed	connect the USB-UART to see which line says FAIL
FPGA: no text in the terminal	adapter TX/RX swapped, no common GND, wrong COM port	adapter RX to JA pin 1, TX to JA pin 2, GND to JA pin 5
Vivado: `net type must be explicitly specified`	file order: `project.v` read before TinyQV	use the provided scripts (they read TinyQV first)
Vivado: `function must have at least one input`	file read as Verilog-2001	read the chip RTL as SystemVerilog (`-sv`)

---

11. Where everything is

Path	What
`src/project.v`, `src/timer.v`, `src/tinyQV/`	chip RTL (top module `tt_um_santhosh_rvsoc`)
`src/config.json`, `info.yaml`	hardening configuration, Tiny Tapeout metadata
`firmware/`	demo and self-test firmware (C + assembly)
`test/`	cocotb testbench and tests (RTL and gate level)
`fpga/rtl/`, `fpga/pynq_z2/`, `fpga/sim/`	FPGA harness, constraints, Vivado scripts, FPGA testbenches
`runs/wokwi/`	the final hardening run (layout in `final/gds/`)
`reports/summary.md`, `reports/sweep.md`	final numbers, iteration history
`reports/precheck/`	Tiny Tapeout precheck reports
`reports/fpga/`	Vivado reports and FPGA simulation logs
`docs/`	this guide, datasheet, I/O guide, FPGA guide, design report, images, PDFs
`scripts/`	environment check, run-all, harden, precheck, gate-level test, docs
`STATUS.md`	project status, decisions and the remaining manual steps

Glossary

Term	Meaning
XIP	execute in place: the CPU runs code straight out of the flash, without copying it
QSPI / QPI	serial memory bus with 4 data lines; QPI sends commands on 4 lines too
PSRAM	pseudo-static RAM, a DRAM with an SRAM-like serial interface
RV32EC	32-bit RISC-V with 16 registers (E) and 16-bit compressed instructions (C)
Nibble-serial	the CPU handles 4 bits per clock, so one 32-bit operation takes 8 clocks
RTL / gate level	the Verilog source / the netlist of real standard cells after layout
cocotb	Python testbench framework driving a Verilog simulator (Icarus here)
xsim	Vivado's built-in simulator
MMCM	clock synthesiser inside the FPGA (125 MHz -> 64 MHz here)
GDS	the final chip layout file sent to the foundry
DRC / LVS	layout rule check / layout-versus-schematic check
Corner (tt/ss/ff)	typical / slow / fast transistor and temperature assumptions in timing analysis